How Laxu uses Privy

Laxu runs on Monad testnet (chain 10143). It turns an open Perpl trade into a token that can be borrowed against. Privy is the identity and wallet layer. It also powers one feature that goes past login: **Protect this loan**. Here Laxu's server can repay part of a user's loan *from the user's own embedded wallet*. It can do this only because Privy lets the user add Laxu's authorization key as a signer, and Privy enforces a policy on everything that signer asks for.

This document lists each Privy feature Laxu uses, where it is in the code, and what evidence exists. Each item is in one of four states:

- **Proven:** a script or an on-chain transaction shows it.
- **Working:** built, and in use in the app.
- **Not exercised:** built, but not yet run end to end.
- **Not shipped:** prototyped as a spike only.

Contents

1. Overview
2. Identifiers
3. Architecture of the integration
4. Feature by feature
5. Considered and not shipped
6. Evidence
7. Screenshots
8. How to reproduce
9. Status and limits

1. Overview

The Privy app is named **Laxu** (read from Privy's `apps.get` API on 2026-10-09). Laxu uses Privy for four things:

1. **Login and embedded wallets.** Users sign in with email, Google or an external wallet. A user who arrives without a wallet gets a Privy embedded wallet at first login.
2. **User-signed transactions.** The user's wallet, embedded or external, signs the user's own transactions through Privy's provider: the AUSD payment that opens a trade, approvals, and every lending call.
3. **Server-side token verification.** Every authenticated backend call carries a Privy access token, and the backend verifies it. The backend gets the user's wallet from

Privy's record of that user, never from the request body.

4. **Signers and policies.** Laxu's server authorization key (a 1-of-1 key quorum) is added as a signer on the user's embedded wallet. The signer carries a policy that allows only `repay()` on one lending pool, up to a per-call cap, with no native value, on chain 10143. Privy checks the policy on every request. This is what **Protect this loan** runs on.

Feature	What it does in Laxu	Where in code	Status
Login	Email, Google, external wallet through Privy's modal	App/src/app/providers.tsx:18	Working
Embedded wallets	Created at login for users without a wallet	App/src/app/providers.tsx:19	Working
User-signed transactions	viem wallet client over the Privy wallet's provider	App/src/lib/walletClient.ts:11	Working; one embedded-wallet payment on chain (section 6)
Access-token verification	`verifyAccessToken` on every authenticated route; 401 on failure	Backend/src/auth/privy.ts:49	Working
Wallet resolved from Privy	The backend reads the user's wallet from Privy, not from the client	Backend/src/auth/privy.ts:99	Working
Signer on a user wallet, with a policy	Server acts on the user's embedded wallet only within the policy	Backend/src/privy/policies.ts:54, App/src/lib/privySigner.ts:19	Proven by script and transaction
Protection worker	Sends `repay()` through Privy's wallet RPC when health falls to the trigger	Backend/src/services/protection.ts:127	Proven by transaction (section 6)
Export wallet, link email or wallet	Account menu	App/src/components/auth/AccountMenu.tsx:344	Built; not exercised in a browser
Privy server wallets	Spike only (section 5)	Backend/scripts/privy/01-server-wallet.ts	Not shipped

2. Identifiers (public values only)

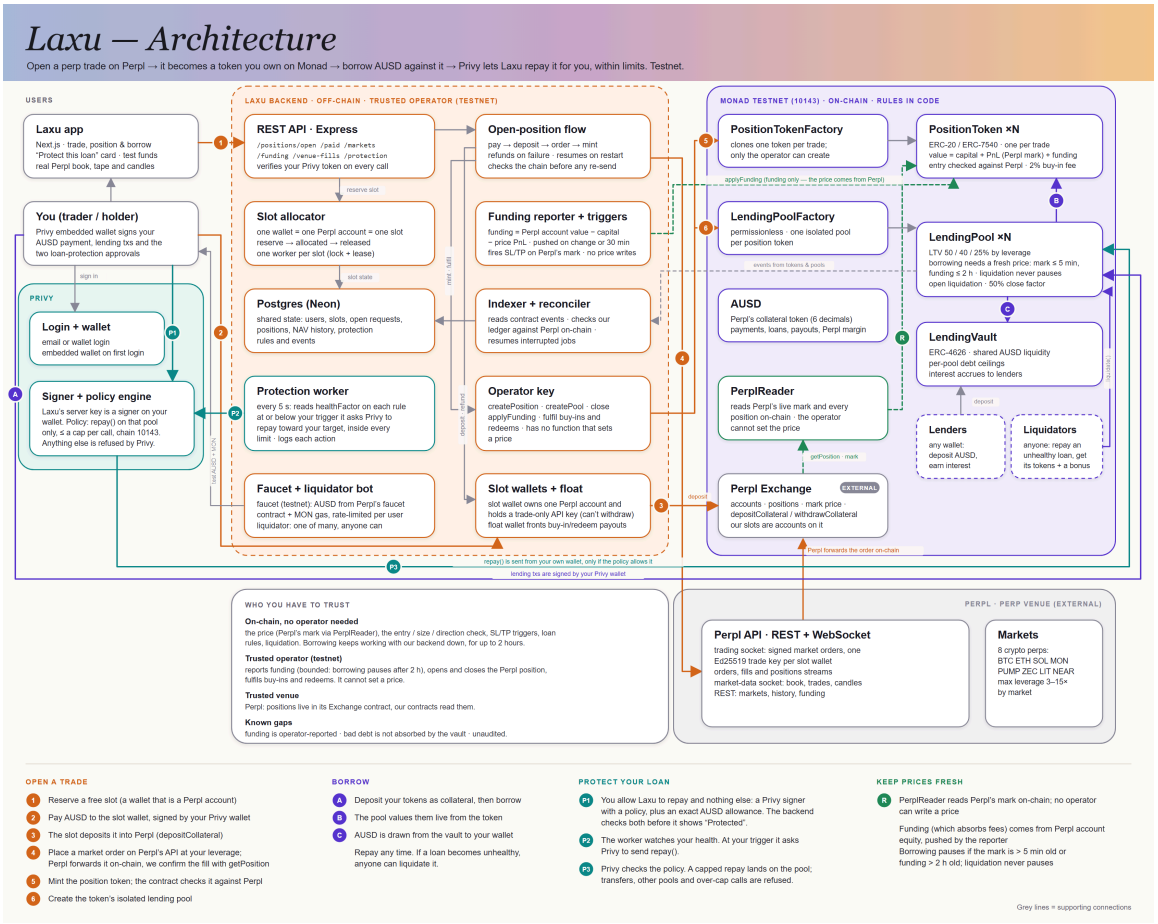
Item	Value	Notes
Privy app name	`Laxu`	Returned by Privy's `apps.get` for this app id on 2026-10-09

Item	Value	Notes
Privy app ID	`cmuvf98y202d10bl8du40p495`	Public client id. Same value in `App/.env.local` (`NEXT_PUBLIC_PRIVY_APP_ID`) and `Backend/.env` (`PRIVY_APP_ID`); the env example files leave it blank
Signer (key quorum) ID	`pwgeso78bs5m8sqm5z24khor`	Value of `PRIVY_SIGNER_ID`. It is an identifier, not a key
Key quorum setup	Display name `laxu-server`, threshold 1, one P-256 authorization key, no user members	Read from Privy's `keyQuorums.get` on 2026-10-09; created by `00-create-signer.ts`
Policy ID (full)	**`wudc3vujs3wri9uoeerv5ws0`**	The policy that was attached to the user signer in the proof run (section 4.4). Named `laxu-spike P1 allow-only`, owned by the quorum above. Read from `docs/privy-findings.md`, `Backend/.e2e/privy-spike.json` and Privy's `policies.get` on 2026-10-09
Loan-protection policy IDs	One per protection rule, created at setup	Built by `buildRepayPolicy`. The rule behind the repay in section 6 uses `peq7eor5iw53mim4nujdwbzs`, named `laxu-repay-0xB3a91c5c`, created 2026-10-10 (screenshot 04)
Chain	Monad testnet, chain ID 10143 (`eip155:10143`)	
Allowed origins	`https://laxu-monad.vercel.app`, `http://localhost:3000`	The hosted frontend, and localhost for testing

The app secret and the authorization private key are never in this repository. They live only in the untracked `Backend/.env`. No script in `Backend/scripts/privy/` prints either one.

The policy id `wudc3vujs3wri9uoeerv5ws0` is a test policy: it allows `approve` up to 1,000 base units on the AUSD token. The proof run used it because it exercises the same machinery as the repay policy (a target address, a function name and a numeric argument limit) without a loan. The policy that protection attaches in production is shown in section 4.4.

3. Architecture of the integration



The signer flow for loan protection:

```
 User's browser                    Laxu backend                  Privy
Monad (10143)
 ──────────────                    ────────────                  ─────
──────────────
 1. POST /protection  ──────────►  validate rule, create policy ─► policies().create
                      ◄──────────  { signerId, policyIds, allowance to approve }
 2. useSigners().addSigners(
      signerId = laxu key quorum,
      policyIds = [this rule's policy]) ──────────────────────────► signer + policy on the
                                                                    user's embedded wallet
 3. approve(pool, maxSpend), signed by the user
──────────────────────────────────────────────────► AUSD allowance
 4. POST /protection/:id/activate ►  ask Privy: our signer, this policy? ─►
wallets().getWalletByAddress
                                    read allowance on chain
──────────────────────────► allowance()
                                    both OK → rule enabled
 ...every 5 s: worker reads healthFactor
──────────────────────────────────────────────────► LendingPool
    health <= trigger:  sendTransaction(repay(amount)),
                        signed with Laxu's authorization key ─► policy check:
                                                                to = pool? chain 10143?
```

```
value 0?                                                    function repay? amount <=
cap?                                                        pass → sign + broadcast
──► repay() from the                                        fail → 400
policy_violation  user's wallet
```

SDKs, at the versions installed (`package.json` and `node_modules`):

Side	Package	Version	Calls
Frontend	`@privy-io/react-auth`	3.45.0 (`^3.45.0`)	`PrivyProvider`, `usePrivy`, `useWallets`, `useLogin`, `getAccessToken`, `useSigners` (`addSigners`, `removeSigners`), `useExportWallet`, `useLinkAccount`
Backend	`@privy-io/node`	0.35.0 (`^0.35.0`)	`utils().auth().verifyAccessToken`, `users()._get`, `wallets().getWalletByAddress`, `wallets().ethereum().sendTransaction`, `policies().create`, `policies().delete`

4. Feature by feature

4.1 Login and embedded wallets

What it is. `PrivyProvider` is configured with `loginMethods: ["email", "wallet", "google"]` and `embeddedWallets: { ethereum: { createOnLogin: "users-without-wallets" } }`. Monad testnet is the default and only chain (App/src/app/providers.tsx:15-22). In the Privy dashboard the app has email, wallet and Google login enabled (`email_auth`, `wallet_auth`, `google_oauth` are all `true` in `apps.get`).

Why Laxu needs it. A trader who signs in with email gets a wallet with no seed phrase to manage. Loan protection needs an embedded wallet, because a signer can only be added to a Privy wallet.

How it works. `SessionProvider` reads `usePrivy()` and `useWallets()` (App/src/lib/session.tsx:52-53). After each login it calls `POST /users/me` once (session.tsx:94). The backend resolves the wallet from Privy's record of the user (Backend/src/auth/privy.ts:99): the external wallet if the user connected one, otherwise the first embedded wallet. That address becomes the user's identity in Laxu. Privy creates the embedded wallet a moment after login, so the backend answers `409 WALLET_NOT_READY` until it exists (Backend/src/services/users.ts:37-40). The frontend retries up to six times, 1.5 s apart. The session's `wallet` is the connected wallet whose address matches the backend's record, never just `wallets[0]` (session.tsx:125-128).

Evidence and limits. Two email users have Privy embedded wallets: `0xfc7d5c97ec539215fab84a732b74f5dce6833d21` (Privy wallet id `ut2hhpkqfbfl2q0637v89o8x`) and `0x8589211269ba098c7e579a116de8c7d12f0b8033`. A user holding both an external and an embedded wallet acts as the external one, and so cannot use loan protection.

4.2 User-signed transactions

What it is. Every on-chain action a user takes goes through a viem wallet client built on the Privy wallet's EIP-1193 provider (App/src/lib/walletClient.ts:11-18). For an embedded wallet, Privy signs. For an external wallet, the user's wallet app signs.

The transactions (all in App/src/lib/actions.ts):

Transaction	Call	Line
Open a trade: pay the slot wallet	AUSD `transfer(payTo, amount)`	actions.ts: 226
Approve before a deposit or buy-in	AUSD or position-token `approve` (`approveIfNeeded`)	actions.ts: 98
Post collateral	`LendingPool.depositCollateral`	actions.ts: 652
Borrow	`LendingPool.borrow`	actions.ts: 703
Repay	`LendingPool.repay`	actions.ts: 710
Withdraw collateral	`LendingPool.withdrawCollateral`	actions.ts: 697
Protection: exact allowance (set, top up, or 0)	AUSD `approve` to exactly the amount (`setExactAllowance`)	actions.ts: 122

Evidence. The embedded wallet `0xfc7d…3d21` paid for a real trade on 2026-10-09 at 16:12:56 UTC: an AUSD `transfer` of 100 AUSD to slot wallet `0x27FAeC53E9fdAe9e4Fac0aF5CC4731e77aE8e503`, tx `0xb5a8e1dd…`. The open request `cmv15z36s0001ofh7pn38kw9u` reached `minted`. At that time no server signer was on the wallet: it was removed at 04:50 UTC that day, and Privy reports `additional_signers: []`. So the user's own session signed it.

Limits. The transactions in docs/e2e-run.md (deposit, borrow, repay, withdraw) were signed by plain script wallets, not Privy wallets. They show the contracts working, not Privy. No embedded wallet has yet signed a deposit, borrow, repay or withdraw on chain.

4.3 Backend access-token verification

What it is. `requireUser` (Backend/src/auth/privy.ts:49-73) reads `Authorization: Bearer <token>` and calls `privy().utils().auth().verifyAccessToken(...)` from `@privy-io/node` (privy.ts:58). The SDK checks the token's signature against the app's verification key. That key is `PRIVY_JWT_VERIFICATION_KEY` when set, and is otherwise fetched from Privy. The verified `user_id` (the Privy DID) is the caller's identity. The frontend attaches the token with `getAccessToken()` (App/src/lib/api.ts:29-31).

Failure. A missing header gives `401 MISSING_TOKEN`. A token that fails verification gives `401 INVALID_TOKEN` (privy.ts:50-63; `unauthorized` is HTTP 401 in Backend/src/lib/errors.ts:16). A valid token for a user who never called `POST /users/me` gets `401 USER_NOT_REGISTERED` on routes that act on a wallet.

Routes behind `requireUser`:

Router	Routes
`/users`	`POST /me`, `GET /me`, `PATCH /me/tag`
`/positions`	`POST /open`, `POST /open/:id/paid`, `GET /open/:id`, `GET /mine`, `GET /:id`
`/faucet`	`GET /status`, `POST /claim`
`/protection`	`POST /`, `POST /:id/activate`, `GET /`, `DELETE /:id`

Why it matters. An address in a request body is never trusted. Protection routes act on the wallet the backend resolved from Privy for the verified user (Backend/src/routes/protection.ts:26-30).

4.4 Signers and policies

Why a signer is needed. `LendingPool.repay(amount)` repays the caller's own debt and pulls AUSD from the caller's wallet. Laxu cannot repay someone's loan from its own wallet. It has to act *as the user*, and only for that one call. A Privy signer with a policy gives exactly that.

The policy. Each protection rule gets its own policy, built by `buildRepayPolicy` (Backend/src/privy/policies.ts:54-82) and created in Privy at setup (Backend/src/services/protection.ts:462). Below is the exact body the code produces, printed by running `buildRepayPolicy` on 2026-10-09. The example inputs are the lending pool from the recorded run and a 50 AUSD per-call cap:


```
{
  "name": "laxu-repay-0x5566777B",
  "version": "1.0",
  "chain_type": "ethereum",
  "rules": [
    {
      "name": "repay up to the cap, on this pool only",
```

```
              "method": "eth_sendTransaction",
              "action": "ALLOW",
              "conditions": [
                { "field_source": "ethereum_transaction", "field": "to", "operator": "eq",
"value": "0x5566777B0635E5185Ee5039634576fDB5c8962Dd" },
                { "field_source": "ethereum_transaction", "field": "chain_id", "operator":
"eq", "value": "10143" },
                { "field_source": "ethereum_transaction", "field": "value", "operator":
"lte", "value": "0x0" },
                { "field_source": "ethereum_calldata", "field": "function_name",
"operator": "eq", "value": "repay",
                  "abi": [{ "name": "repay", "type": "function", "stateMutability":
"nonpayable",
                            "inputs": [{ "name": "amount", "type": "uint256" }],
"outputs": [{ "name": "repaid", "type": "uint256" }] }] },
                { "field_source": "ethereum_calldata", "field": "repay.amount",
"operator": "lte", "value": "0x2faf080",
                  "abi": [{ "name": "repay", "type": "function", "stateMutability":
"nonpayable",
                            "inputs": [{ "name": "amount", "type": "uint256" }],
"outputs": [{ "name": "repaid", "type": "uint256" }] }] }
              ]
            }
          ],
          "owner_id": "pwgeso78bs5m8sqm5z24khor"
        }
```

What each condition means:

Condition	Plain words
`method: eth_sendTransaction`	Only a request to send a transaction. Signing messages or typed data does not match
`to eq <pool>`	Only to this one lending pool, never the AUSD token, another pool or any other address
`chain_id eq 10143`	Only on Monad testnet
`value lte 0x0`	No MON may move with the call
`function_name eq repay`	The calldata must decode as `repay(uint256)`. `approve`, `transfer`, `borrow` and `withdrawCollateral` do not match
`repay.amount lte 0x2faf080`	At most the per-call cap (50,000,000 base units = 50 AUSD here). The cap is half the user's maximum spend (Backend/src/services/protectionRules.ts:85)

There is **one ALLOW rule and no DENY rule**, on purpose. The spike measured two things. A request that matches no rule is denied. And a DENY-all rule overrides the ALLOW in either order, so it would block every repay (docs/privy-findings.md, 1.2). A test pins the single-rule shape (Backend/src/privy/policies.test.ts:51). Privy also refuses a rule name of 50 characters or more. A real run caught that, and a test now pins the length (policies.test.ts:45).

The rule is narrow because the signer has no other reason to exist. Whatever it can do, someone holding Laxu's server key could do. With this policy the worst that key can do is repay the user's own debt on one pool, in steps no larger than the cap.

Who holds what. The server holds one P-256 authorization key, registered with Privy as the 1-of-1 key quorum `pwgeso78bs5m8sqm5z24khor`. Its private half is `PRIVY_AUTH_PRIVATE_KEY`, kept in `Backend/.env` and normalised to the base64 PKCS8 form the SDK expects (Backend/src/privy/authKey.ts:11). The user's embedded wallet stays owned by the user. Laxu never holds a user key. Adding the signer makes the quorum an *additional* signer on the wallet, restricted by the policy id passed with it. When the worker sends, the request is signed with the server key alone (`authorization_context`, Backend/src/privy/signer.ts:43-57). Privy accepts it only because the quorum is a signer on that wallet, and only if the policy passes.

Adding and removing the signer. The card calls `useSigners()` through a thin wrapper (App/src/lib/privySigner.ts:19-43):

- `addSigners({ address, signers: [{ signerId, policyIds: [policyId] }] })` (privySigner.ts:27)
- `removeSigners({ address })` (privySigner.ts:35), which removes every signer from that wallet

No Privy popup appears for `addSigners` on this setup. This was observed in both proof runs, on 2026-10-08 and 2026-10-09. So the app supplies the consent step itself. Before anything is granted, the setup panel shows the consent text (App/src/components/position/ProtectionParts.tsx:148-150):

> Laxu will be allowed to call `repay` on this loan's pool from your wallet, only when your health drops to *{trigger}*, for at most *{per call}* per call and *{total}* in total. It cannot send your funds anywhere else. You can turn this off at any time.

The button under it reads **"I understand, allow Laxu to repay"** (App/src/components/position/ProtectionSetup.tsx:271). Nothing is granted before that press. The card says "Powered by Privy" next to the Privy step, and never shows or implies a Privy dialog.

The card's states (App/src/components/position/ProtectionCard.tsx). The backend computes a `phase` for each rule (`setup`, `on` or `off`), and the card renders from it:

State	When	What the user sees	Line
Off	No rule, or a rule turned off with nothing left behind	"Set up protection" (disabled with "Borrow first" when there is no debt)	ProtectionCard.tsx:313
Setup	Form open	Trigger, target, max spend, repay preview, consent text, two steps	ProtectionCard.tsx:231

State	When	What the user sees	Line
Finish setup	A rule in phase `setup` (for example after a reload mid-setup)	"Setup not finished. Pick up where you left off, or cancel it." Resumes from the live signer and allowance	ProtectionSetup.tsx:277
On	Phase `on` (the backend has verified both grants)	**PROTECTED** badge, health, limits, events, Turn off	ProtectionCard.tsx:247
Cleanup needed	Phase `off`, but the signer or an allowance is still on the wallet	"Protection is off and Laxu cannot act, but the permission is still on your wallet." with buttons to remove each one	ProtectionCard.tsx:289

Two grants, and why both. The setup steps are fixed: "Step 1 of 2: Allow Laxu to repay" (the signer), then "Step 2 of 2: Approve spending limit" (the allowance) (ProtectionSetup.tsx:67-69).

- The **signer** (Privy side) limits *what* Laxu can call: `repay` on that pool, up to the per-call cap.
- The **allowance** (chain side) limits *how much* Laxu can ever spend in total. The user approves exactly their maximum spend to the pool, in their own transaction (`setExactAllowance`, never max-uint).

The policy has no total limit. Privy's cumulative limits apply to `eth_signTransaction`, not to `eth_sendTransaction`, so the ERC-20 allowance is the total cap. Before a rule turns on, `activateRule` checks both (Backend/src/services/protection.ts:504-545). It asks Privy whether the wallet carries Laxu's signer with exactly this rule's policy and nothing else (protection.ts:517-524). It reads the allowance on chain (protection.ts:526-531). It refuses with `SIGNER_MISSING`, `SIGNER_POLICY_MISMATCH` or `ALLOWANCE_TOO_LOW` otherwise. The card shows "Protected" only after the backend says so.

Turn-off order (ProtectionCard.tsx:194-205):

1. **Backend disable** (`DELETE /protection/:id`). The worker stops at once, and this step needs nothing from Privy.
2. **Revoke the signer** (`removeSigners`).
3. **Allowance to 0** (`approve(pool, 0)`).

If step 2 or 3 is rejected, the rule lands in Cleanup needed, which offers only the steps that remain.

Evidence: the signer acting under the policy. Script Backend/scripts/privy/03-user-signer.ts, run 2 on 2026-10-09, used the embedded wallet `0xfc7d…3d21` and policy `wudc3vujs3wri9uoeerv5ws0` (`approve` on AUSD up to 1,000). The user pressed Add signer on the dev page at 04:47:59 UTC. The script then ran with only the server's key, no user signature:

```
$ cd Backend
$ npx ts-node --transpile-only scripts/privy/03-user-signer.ts --wallet
0xfc7d5c97ec539215fab84a732b74f5dce6833d21
additional_signers: 1; ours present: true; its policy ids: ["wudc3vujs3wri9uoeerv5ws0"]
```

Call from the user's wallet, signed only by the server key	Result
`approve(0x…dEaD, 1000)` (at the cap)	**Allowed.** Mined: `0x370e8a29…`, `from` = the user's wallet, status `success`
`approve(0x…dEaD, 1001)` (over the cap)	**Refused:** `policy_violation`
`transfer(0x…dEaD, 1)` on the same token (wrong function)	**Refused:** `policy_violation`
`approve` sent to another address (wrong target)	**Refused:** `policy_violation`

Privy's refusal body, the same every time: `HTTP 400 {"error":"RPC request denied due to policy violation","code":"policy_violation"}`.

The user then pressed Remove signer (04:50:19 UTC), and the script ran again with `--expect-removed`. It reported `additional_signers: 0`, and the same allowed call failed. Verbatim from `Backend/.e2e/privy-spike.json`:

```
NOT POLICY HTTP 401 {"error":"No valid authorization signatures were provided. Your
payload may be malformed or your signing keys may be incorrect or expired. Docs:
https://docs.privy.io/api-reference/authorization-signatures"}
```

These lines and results are as recorded in docs/privy-findings.md (1.3, run 2); the full terminal transcript was not saved.

4.5 Protection worker

Loop. With `ENABLE_PROTECTION=true`, the backend refuses to boot if the Privy credentials, signer id or authorization key are missing. It then starts `startProtectionJob` (Backend/src/services/protection.ts:138). Every `PROTECTION_INTERVAL_MS` (default 5,000 ms, Backend/src/config/env.ts:161), `runProtectionTick` (protection.ts:127) checks each enabled rule. For each one it reads `healthFactor` and `currentDebt` from the pool. At or below the trigger, and outside the 60 s cooldown, it plans a repay and sends it.

Repay amount (Backend/src/services/protectionMath.ts:54-92):

```
healthFactor = collateralValue × thresholdBps × 1e18 / (10,000 × debt)
D'           = collateralValue × thresholdBps × 1e18 / (10,000 × targetHealth)
debtAtHealth, rounded down
base         = debt − D'
needed       = min(base + ceil(base × 100 / 10,000), debt)
```

```
REPAY_BUFFER_BPS = 100 (1%)
amount       = min(needed, maxPerCall, remainingSpend, balance, allowance)
send nothing if amount < DUST_FLOOR (10,000 base units = 0.01 AUSD)
```

The clamps are named `maxPerCall`, `remainingSpend`, `balance` and `allowance`, in that order. A plan records which clamp cut it, so the card can tell the user which limit stopped a full repay.

Sending. `executeRepay` (protection.ts:285-342) writes a `PENDING` event, estimates gas with Laxu's own RPC plus a buffer (Monad bills the gas *limit*), and calls `sendTransaction` with `caip2: "eip155:10143"`. It then records what the pool actually took, from the `Repaid` log, as a `REPAID` event with the tx hash and the health before and after. Privy errors are classified (Backend/src/privy/signer.ts:85-92): `policy_violation` maps to policy, 401 to signer removed, and `transaction_broadcast_failure` to broadcast.

Gas. The repay is sent from the user's wallet, so the user's wallet pays the MON gas. Before sending, `gasGate` (protection.ts:271) checks that the wallet can cover the gas limit at the fee cap. If not, the worker records `SKIPPED` with "Add MON for gas" and makes no Privy call.

Every action is logged in `protection_events`, with these kinds: `CREATED`, `ENABLED`, `SKIPPED`, `PENDING`, `REPAID`, `FAILED` and `DISABLED`. The card lists them (App/src/components/position/ProtectionEvents.tsx). A skip note is written at most once per cooldown window.

Tests. All 31 passed on 2026-10-09: `npx tsx --test src/privy/policies.test.ts src/services/protectionMath.test.ts src/services/protectionRules.test.ts`.

File	Tests	Covers
Backend/src/privy/policies.test.ts	5	Exact policy shape (:9); rule name under 50 characters (:45); one ALLOW, no DENY, repay-only ABI (:51); cap is the hex of the base-unit amount, pool changes only the target (:61); non-address refused (:72)
Backend/src/services/protectionMath.test.ts	9	Health parse and format (:32); `debtAtHealth` (:41); exact target (:47); 1% buffer (:57); never more than the debt (:68); each clamp binds on its own and is named (:82); zero results (:108); dust floor (:125); decimals (:141)
Backend/src/services/protectionRules.test.ts	17	Input validation; trigger at exactly the threshold acts (:129); cooldown (:147); skip notes; PENDING handling (:197); gas gate on the limit (:206); re-enable starts clean (:224)
App/scripts/checkProtectionMath.mjs	6 + 9	The App's repay preview matches the backend's `planRepay` on 6 vectors; 9 eligibility cases (external wallet gives `not-embedded`)

5. Considered and not shipped

Privy server wallets for the faucet and liquidator. Script Backend/scripts/privy/01-server-wallet.ts created a Privy server wallet owned by the key quorum (`0x8d69008cb420C27435d6a707206944d828258094`) and sent from it on Monad testnet on 2026-10-08.

What the spike proved:

- Privy signs **and broadcasts** on chain 10143: 5 of 5 sends, median 1,172 ms. Privy set the nonce and gas itself. First hash `0x937f4804…`, gas used 26,456.
- Sign-only plus a viem broadcast also works: `0x107adeb3…`.
- With a policy on that wallet (Backend/scripts/privy/02-policy.ts), Privy enforces `to`, `value`, the function name and a numeric argument limit, and denies by default. Allowed `approve`: `0xb19e166e…`. Five forbidden variants (over the limit, wrong function, wrong address, plain value send, `approve` with value) were refused with `policy_violation`.

The shipped protection signer relies on these results: it uses `sendTransaction` (Privy signs and broadcasts), and it keeps the allow-only policy shape.

The running product does not use server wallets: the faucet, liquidator, slot, float and operator wallets keep their own keys, by the project owner's choice. Privy sits only where the *user's* wallet acts and the user stays in control.

6. Evidence

Transactions

Each hash below was checked with `eth_getTransactionReceipt` and `eth_getTransaction` on 2026-10-09, except the repay, which was checked on MonadScan on 2026-10-13 (screenshot 10). All are `success`.

Step	Who signed	Tx hash	What it proves
Open-trade payment, 100 AUSD `transfer` to slot wallet `0x27FA…e503`	User, in the browser, with the Privy embedded wallet `0xfc7d…3d21` (no server signer on the wallet then)	`0xb5a8e1ddc51c88f0426030f33093360ab080c652d0892bc9403979e836e3e8ad`	The embedded wallet signs the user's real transactions; the open request reached `minted`
`approve(dead, 1000)` from the user's wallet	Laxu's server key, as a signer on the user's embedded wallet, through Privy's wallet RPC under policy `wudc3vujs3wri9uoeerv5ws0`	`0x370e8a29a2c5a29fbdfc9a99db1b2b5b7b745e195ab81a1f64af367cb2f7cc8f`	The server can act on a user's wallet only through the signer, within the policy (`from` = `0xfc7d…3d21`)

Step	Who signed	Tx hash	What it proves
`approve(dead, 1000)` from a Privy server wallet (policy run 2)	Server key, Privy server wallet `0x8d69…8094`, under an allow-only policy	`0xb19e166e065827925daf87e712a5ca181d1794c1c011441e05336680b761c8f9`	The allowed side of the policy test (spike, section 5)
1 wei send, Path A	Privy (`sendTransaction`), server wallet `0x8d69…8094`	`0x937f4804c3f4e2221cfde6920444a8a07a05806df9e18a87d7fb918e8e050ef9`	Privy signs and broadcasts on Monad testnet (spike)
Protection `repay()` on pool `0xB3a9…d328`, 2026-10-13 07:12:04 UTC, block 70619612	Laxu's server key, as a signer on the user's embedded wallet `0xfc7d…3d21`, through Privy's wallet RPC under policy `peq7eor5iw53mim4nujdwbzs`	`0x882f5ef469f9f72ae7e16dada69ad39dfbbefcf127a3b26be370daf18465e099`	Loan protection end to end: the worker fired, and Privy signed and broadcast `repay()` from the user's own wallet, within the repay-only policy (screenshot 10)

Policy refusal log

Refusals are not mined, so they have no hash. Privy answered `HTTP 400 {"error":"RPC request denied due to policy violation","code":"policy_violation"}` for each:

Run	Call	Result
User signer, 2026-10-09 (`03-user-signer.ts`)	`approve(dead, 1001)`	`policy_violation`
User signer, 2026-10-09	`transfer(dead, 1)`	`policy_violation`
User signer, 2026-10-09	`approve` at another address	`policy_violation`
User signer, after removal	`approve(dead, 1000)`	`HTTP 401`, no valid authorization signature
Server wallet, 2026-10-08 (`02-policy.ts`, run 2)	Over the limit, wrong function, wrong address, plain value send, `approve` with value	`policy_violation` ×5

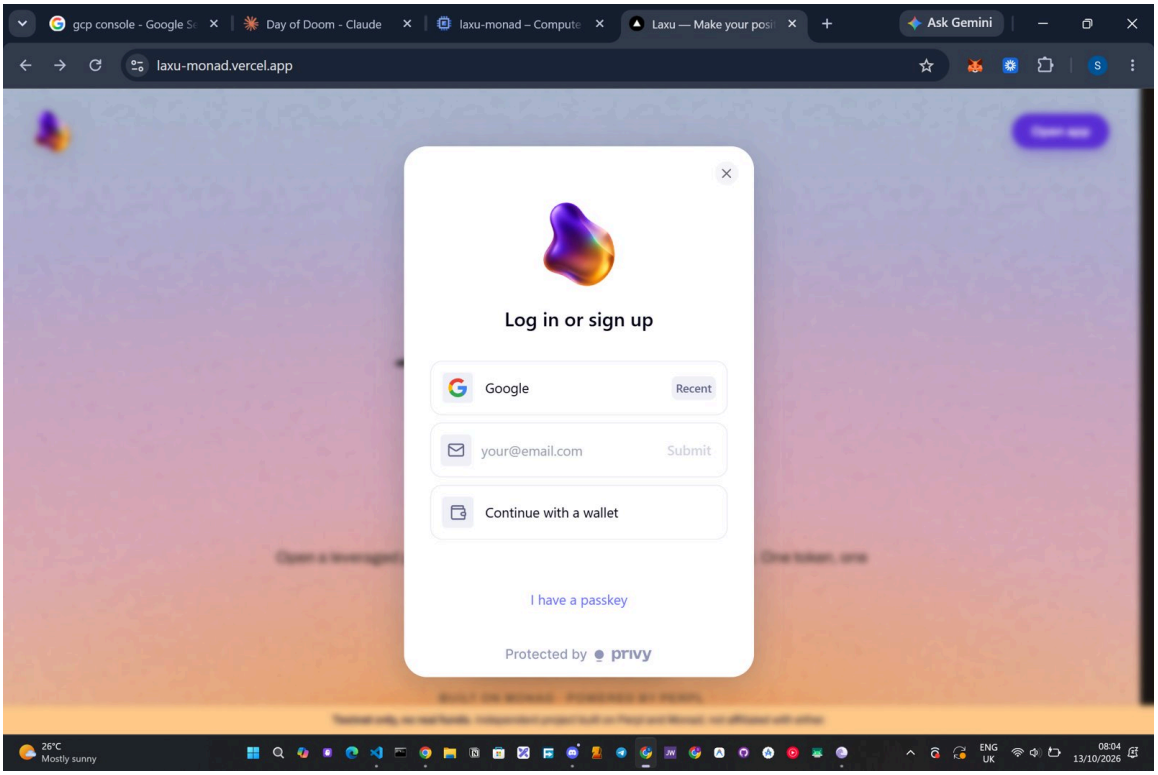
Tests

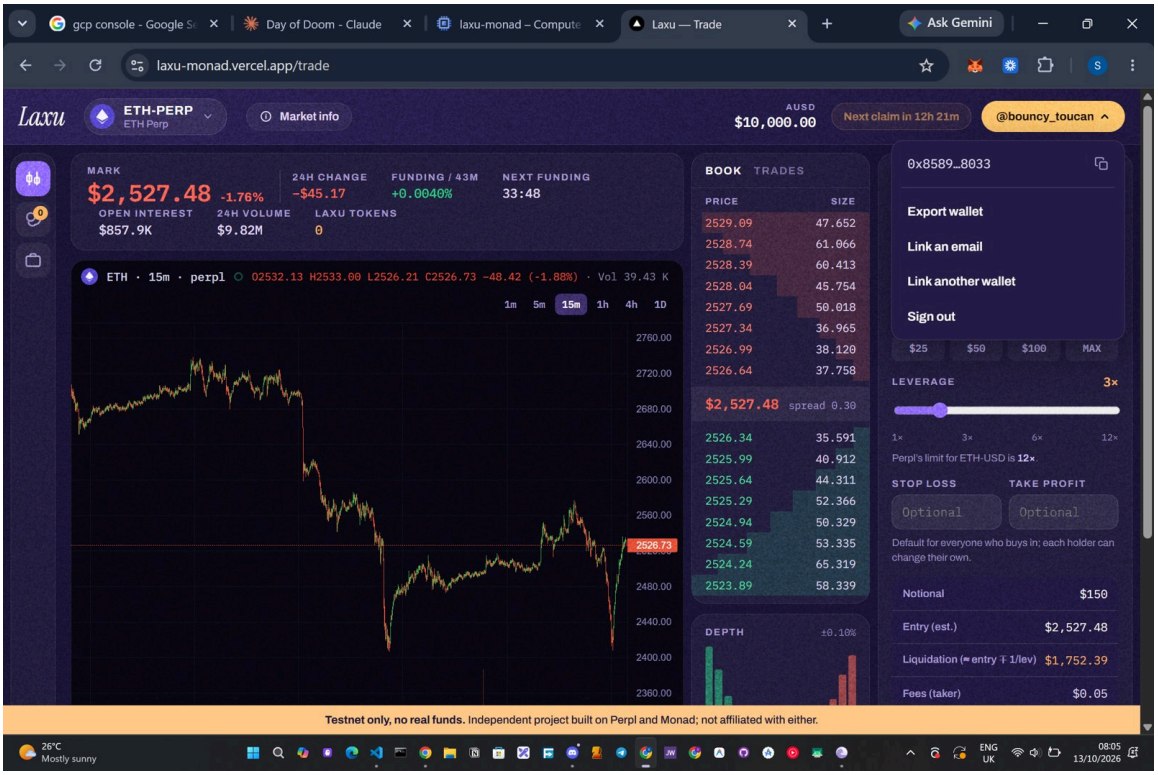
The unit tests are listed in section 4.5. Separately, `Backend/scripts/privy/qa-protection.ts` passed 22 of 22 checks on 2026-10-09 against the real database, chain and Privy, sending no transaction (`Backend/.e2e/privy-qa.json`). They cover validation refusals, the embedded-wallet check, `activate` refused with `SIGNER_MISSING`, other users locked out, and the worker's no-debt and wallet-mismatch paths.

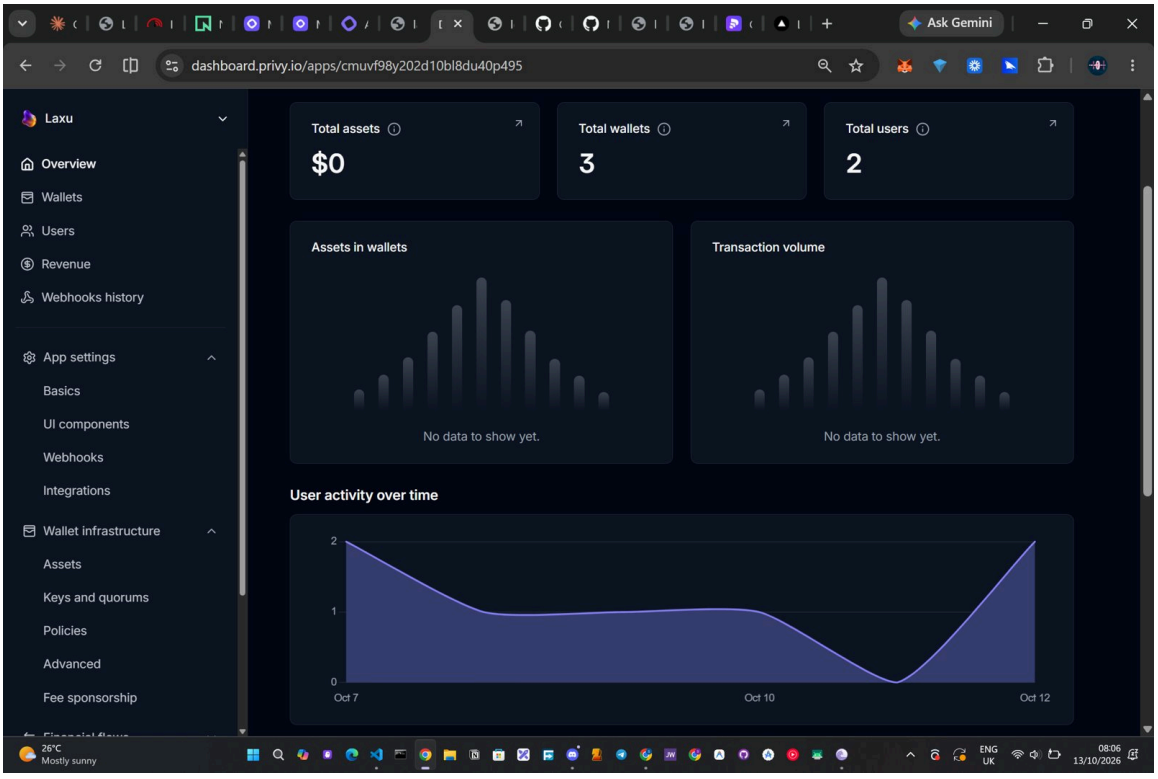
7. Screenshots

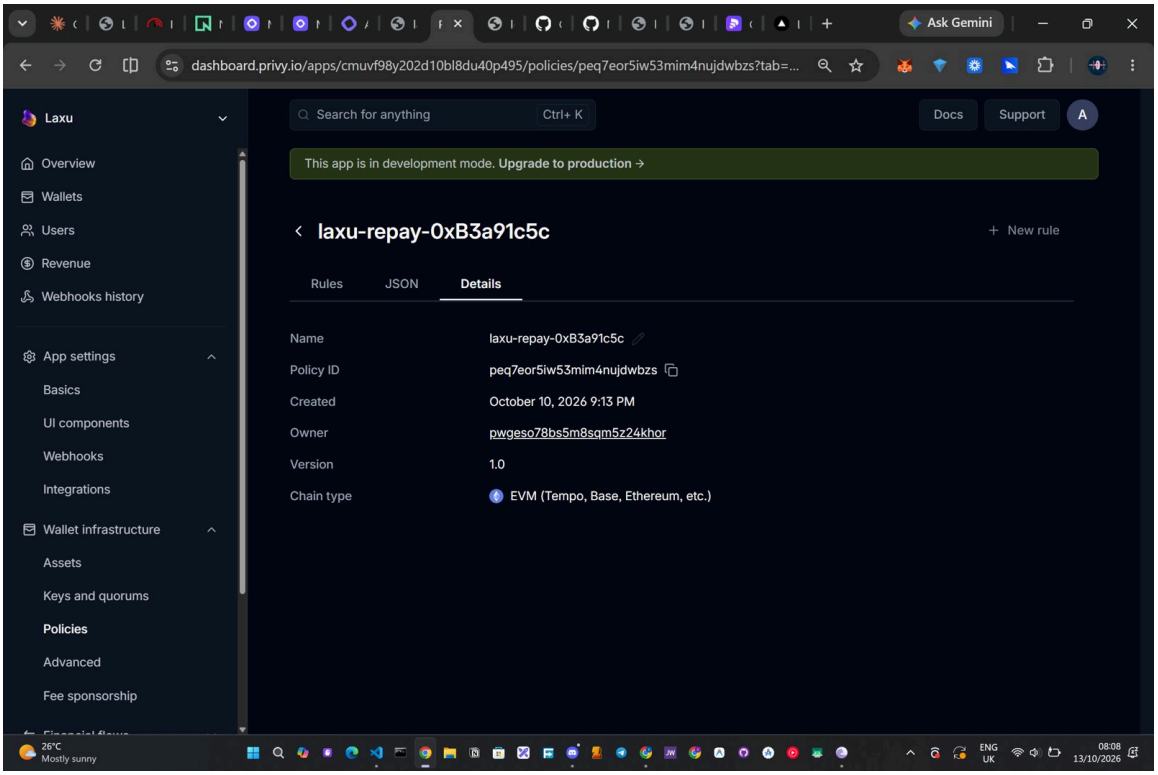
The images are in `docs/img/privy/`, captured on 2026-10-13 from the hosted app (`laxu-monad.vercel.app`) and the Privy dashboard.

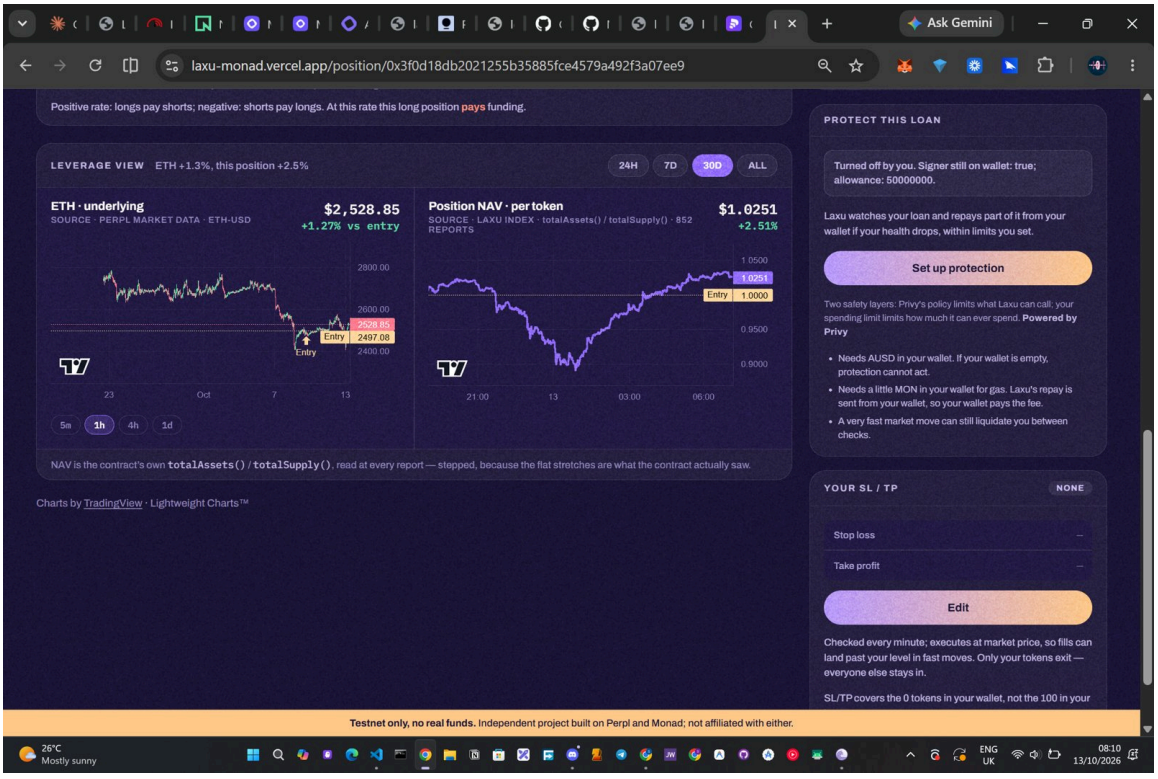
File	What was captured
`01-login.png`	Login modal (email or wallet)
`02-embedded-wallet.png`	Embedded wallet address shown in the app
`03-privy-dashboard-app.png`	Privy dashboard showing the app named Laxu (no app secret visible)
`04-dashboard-policy.png`	The repay policy `laxu-repay-0xB3a91c5c` in the dashboard (full policy id visible)
`06-protect-card-off.png`	"Protect this loan" card, state Off, before setup
`07-protect-setup.png`	Privy's prompt to approve the exact AUSD allowance for the pool, from the embedded wallet
`08-protect-consent.png`	Privy's "Transaction complete" after that approval
`09-protect-on.png`	State On, "Protected"
`10-signed-tx.png`	MonadScan page of the protection repay `0x882f5ef4…`, sent from the embedded wallet by Laxu's signer

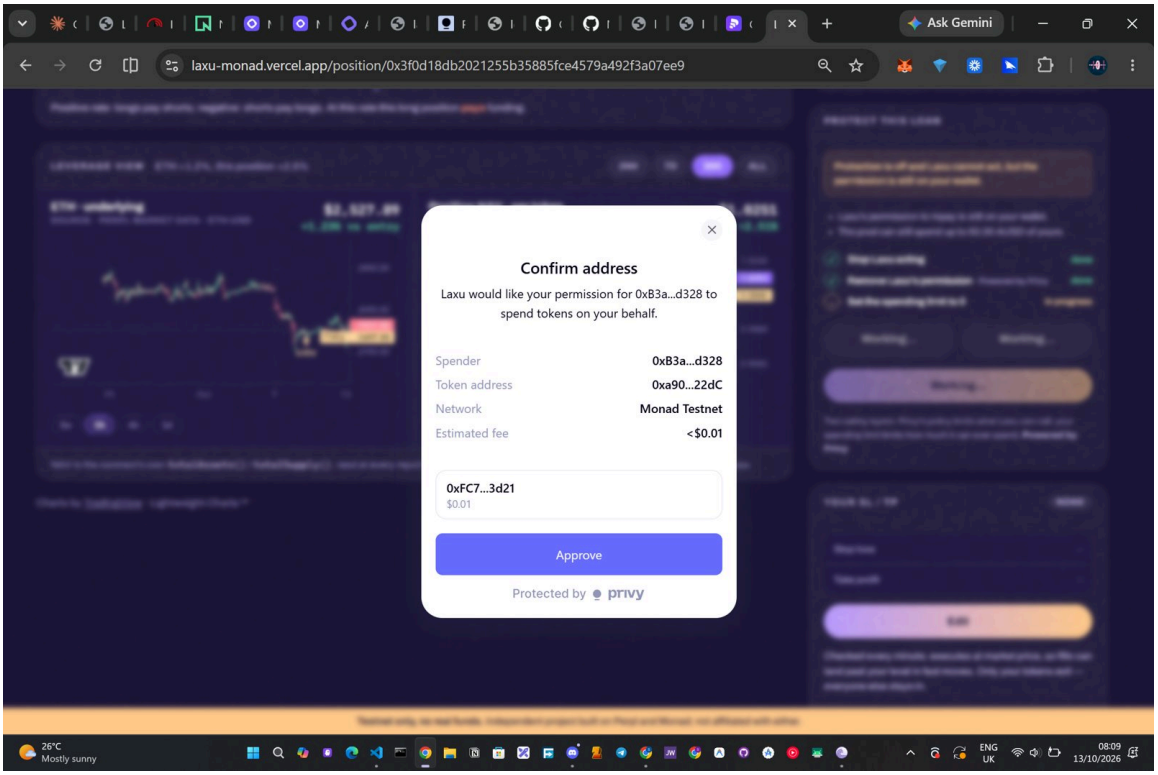


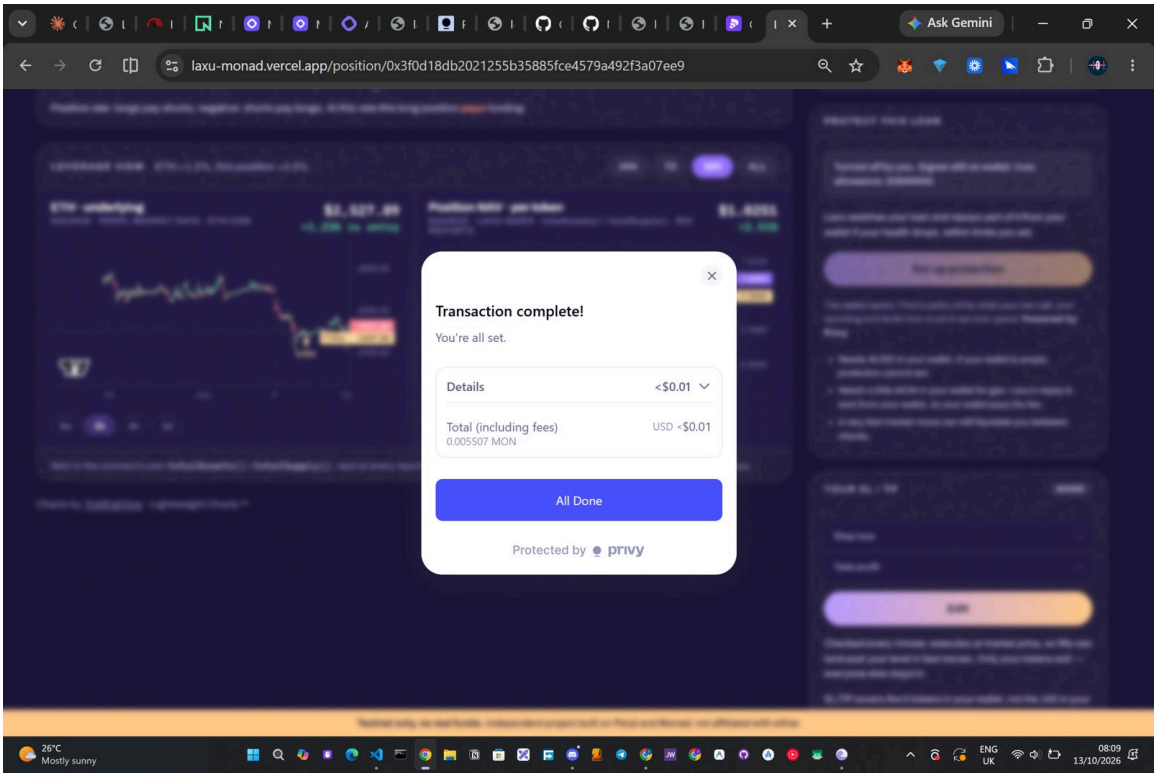

gcp console - Google S
Day of Doom - Claude
laxu-monad – Compute
Laxu — Trade
Ask Gemini
laxu-monad.vercel.app/trade
Laxu
ETH-PERP
ETH Perp
Market info
AUSD
$10,000.00
Next claim in 12h 21m
@bouncy_toucan
MARK
$2,527.48
-1.76%
24H CHANGE
-$45.17
FUNDING / 43M
+0.0040%
NEXT FUNDING
33:48
OPEN INTEREST
$857.9K
24H VOLUME
$9.82M
LAXU TOKENS
0
ETH · 15m · perpl
O2532.13 H2533.00 L2526.21 C2526.73 −48.42 (−1.88%) · Vol 39.43 K
1m
5m
15m
1h
4h
1D
2760.00
2720.00
2680.00
2640.00
2600.00
2560.00
2526.73
2480.00
2440.00
2400.00
2360.00
BOOK
TRADES
PRICE
SIZE
2529.09 47.652
2528.74 61.066
2528.39 60.413
2528.04 45.754
2527.69 50.018
2527.34 36.965
2526.99 38.120
2526.64 37.758
$2,527.48 spread 0.30
2526.34 35.591
2525.99 40.912
2525.64 44.311
2525.29 52.366
2524.94 50.329
2524.59 53.335
2524.24 65.319
2523.89 58.339
DEPTH
±0.10%
0x8589…8033
Export wallet
Link an email
Link another wallet
Sign out
$25
$50
$100
MAX
LEVERAGE
3×
1×
3×
6×
12×
Perpl's limit for ETH-USD is 12×
STOP LOSS
TAKE PROFIT
Optional
Optional
Default for everyone who buys in; each holder can change their own.
Notional
$150
Entry (est.)
$2,527.48
Liquidation (≈ entry ∓ 1/lev)
$1,752.39
Fees (taker)
$0.05
Testnet only, no real funds. Independent project built on Perpl and Monad; not affiliated with either.
26°C
Mostly sunny
ENG
UK
08:05
13/10/2026

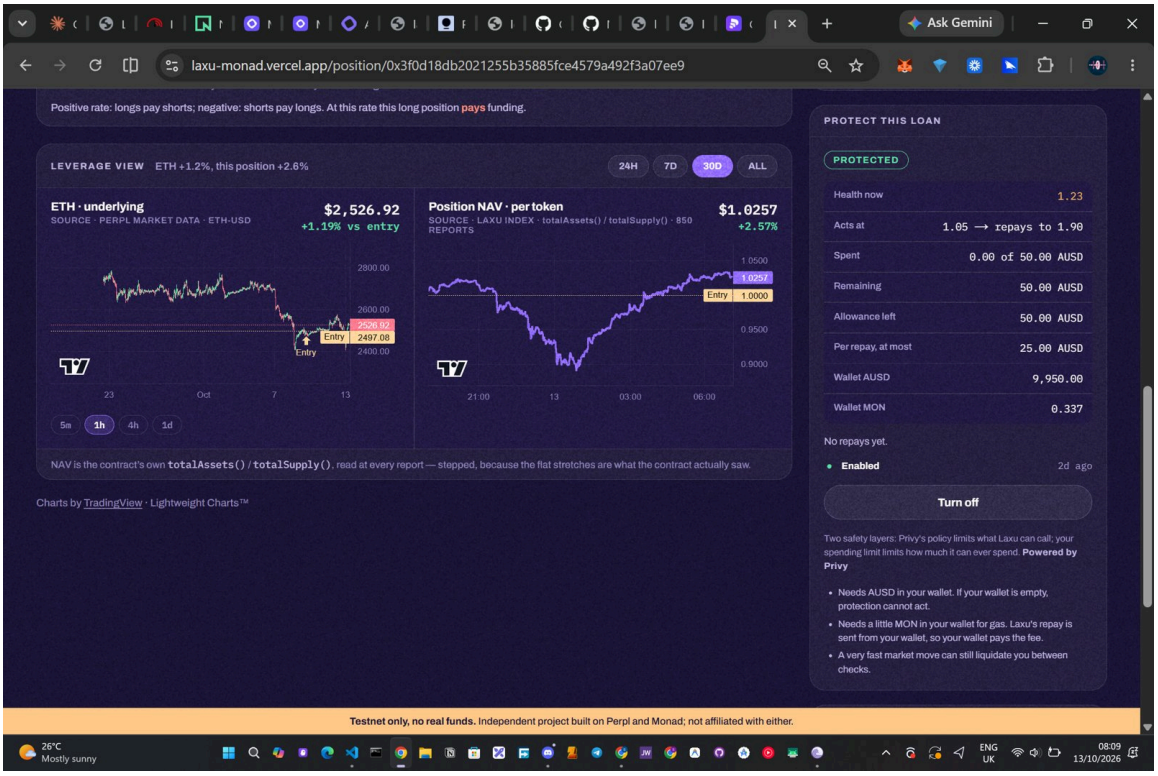

Ask Gemini
dashboard.privy.io/apps/cmuvf98y202d10bl8du40p495
Laxu
Overview
Wallets
Users
Revenue
Webhooks history
App settings
Basics
UI components
Webhooks
Integrations
Wallet infrastructure
Assets
Keys and quorums
Policies
Advanced
Fee sponsorship
Total assets
$0
Total wallets
3
Total users
2
Assets in wallets
No data to show yet.
Transaction volume
No data to show yet.
User activity over time
2
1
0
Oct 7
Oct 10
Oct 12
26°C
Mostly sunny
ENG
UK
08:06
13/10/2026

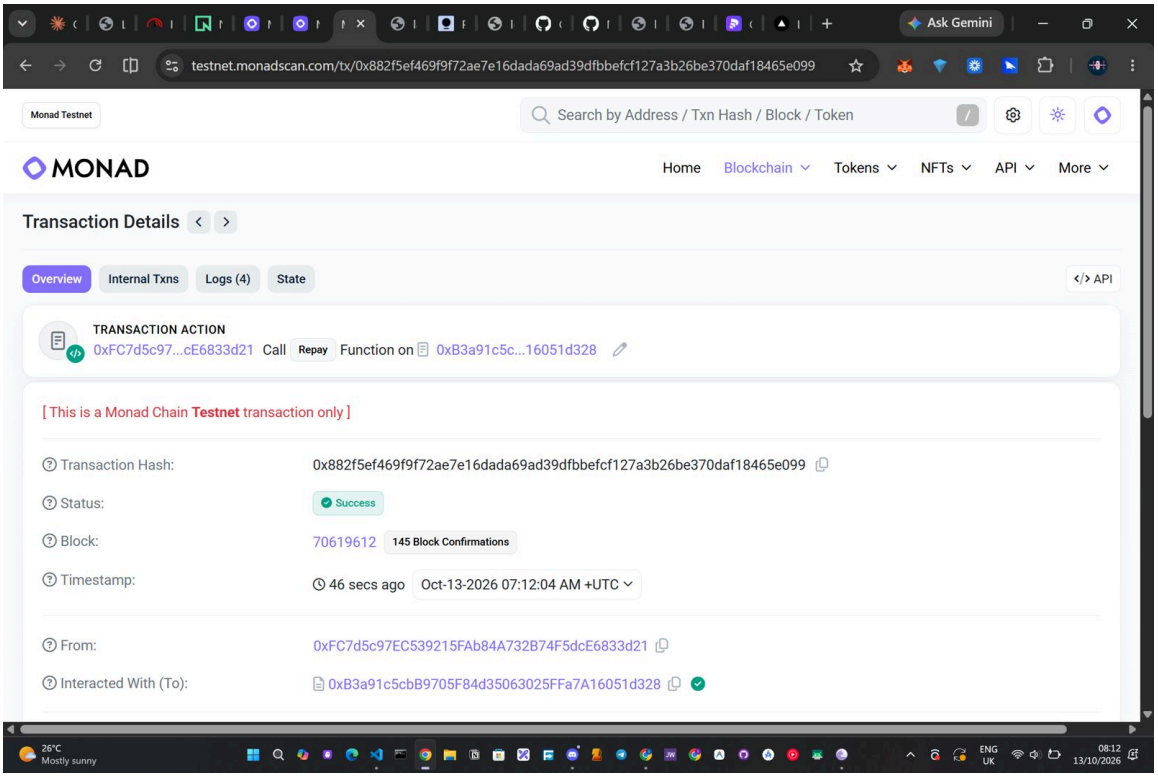

Ask Gemini
dashboard.privy.io/apps/cmuvf98y202d10bl8du40p495/policies/peq7eor5iw53mim4nujdwbzs?tab=...
Laxu
Search for anything
Ctrl+ K
Docs
Support
A
Overview
Wallets
Users
Revenue
Webhooks history
App settings
Basics
UI components
Webhooks
Integrations
Wallet infrastructure
Assets
Keys and quorums
Policies
Advanced
Fee sponsorship
This app is in development mode. Upgrade to production →
laxu-repay-0xB3a91c5c
New rule
Rules
JSON
Details
Name
laxu-repay-0xB3a91c5c
Policy ID
peq7eor5iw53mim4nujdwbzs
Created
October 10, 2026 9:13 PM
Owner
pwgeso78bs5m8sqm5z24khor
Version
1.0
Chain type
EVM (Tempo, Base, Ethereum, etc.)
26°C
Mostly sunny
ENG
UK
08:08
13/10/2026


Ask Gemini
laxu-monad.vercel.app/position/0x3f0d18db2021255b35885fce4579a492f3a07ee9
Positive rate: longs pay shorts; negative: shorts pay longs. At this rate this long position pays funding.
LEVERAGE VIEW
ETH +1.3%, this position +2.5%
24H
7D
30D
ALL
ETH · underlying
SOURCE · PERPL MARKET DATA · ETH-USD
$2,528.85
+1.27% vs entry
2800.00
2600.00
2528.85
Entry
2497.08
Entry
2400.00
23
Oct
7
13
5m
1h
4h
1d
Position NAV · per token
SOURCE · LAXU INDEX · totalAssets() / totalSupply() · 852 REPORTS
$1.0251
+2.51%
1.0500
1.0251
Entry
1.0000
0.9500
0.9000
21.00
13
03.00
06.00
NAV is the contract's own totalAssets() / totalSupply(), read at every report — stepped, because the flat stretches are what the contract actually saw.
Charts by TradingView · Lightweight Charts™
PROTECT THIS LOAN
Turned off by you. Signer still on wallet: true; allowance: 50000000.
Laxu watches your loan and repays part of it from your wallet if your health drops, within limits you set.
Set up protection
Two safety layers: Privy's policy limits what Laxu can call; your spending limit limits how much it can ever spend. Powered by Privy
Needs AUSD in your wallet. If your wallet is empty, protection cannot act.
Needs a little MON in your wallet for gas. Laxu's repay is sent from your wallet, so your wallet pays the fee.
A very fast market move can still liquidate you between checks.
YOUR SL / TP
NONE
Stop loss
Take profit
Edit
Checked every minute; executes at market price, so fills can land past your level in fast moves. Only your tokens exit — everyone else stays in.
SL/TP covers the 0 tokens in your wallet, not the 100 in your
Testnet only, no real funds. Independent project built on Perpl and Monad; not affiliated with either.
26°C
Mostly sunny
ENG
UK
08:10
13/10/2026


Ask Gemini
laxu-monad.vercel.app/position/0x3f0d18db2021255b35885fce4579a492f3a07ee9
Confirm address
Laxu would like your permission for 0xB3a...d328 to spend tokens on your behalf.
Spender
0xB3a...d328
Token address
0xa90...22dC
Network
Monad Testnet
Estimated fee
<$0.01
0xFC7...3d21
$0.01
Approve
Protected by privy
26°C
Mostly sunny
ENG
UK
08:09
13/10/2026


Ask Gemini
laxu-monad.vercel.app/position/0x3f0d18db2021255b35885fce4579a492f3a07ee9
Transaction complete!
You're all set.
Details
<$0.01
Total (including fees)
0.005507 MON
USD <$0.01
All Done
Protected by privy
26°C
Mostly sunny
ENG
UK
08:09
13/10/2026





8. How to reproduce

Steps marked **(your credentials)** need your own Privy app.

1. **Create a Privy app (your credentials).** In the dashboard, enable email login (protection needs an embedded wallet), and optionally Google and wallet login. Add your app's origin to the allowed origins.

2. **Set the env var names.** Values come from your dashboard; never commit them.
   - `Backend/.env`: `PRIVY_APP_ID`, `PRIVY_APP_SECRET`, optional `PRIVY_JWT_VERIFICATION_KEY`; for protection, `PRIVY_SIGNER_ID` and `PRIVY_AUTH_PRIVATE_KEY`; for the spike scripts only, `PRIVY_TEST_CONTRACT`.
   - `App/.env.local`: `NEXT_PUBLIC_PRIVY_APP_ID`; for the dev signer page only, `NEXT_PUBLIC_PRIVY_SIGNER_ID` and `NEXT_PUBLIC_DEV_TOOLS=1`.
3. **Create the authorization key (your credentials):** `cd Backend && npx ts-node --transpile-only scripts/privy/00-create-signer.ts`. It registers a new P-256 public key as a 1-of-1 quorum `laxu-server` and writes `PRIVY_SIGNER_ID` and `PRIVY_AUTH_PRIVATE_KEY` into `Backend/.env`, with a backup at `~/privy-signer-backup.txt`. It prints the quorum id, never the key, and never overwrites.
4. **Create a test policy (your credentials):** `npx ts-node --transpile-only scripts/privy/02-policy.ts` prints an allow-only `approve` policy id. Never attach one made with `--with-deny-all`.
5. **Add the signer.** Start the App with `NEXT_PUBLIC_DEV_TOOLS=1`, open `/dev/privy`, sign in **with email**, paste the policy id, click **Add signer**. Send the embedded wallet about 0.01 MON for gas.
6. **Run the proof at once:** `npx ts-node --transpile-only scripts/privy/03-user-signer.ts --wallet 0xYOUR_EMBEDDED_WALLET`. Expect `ours present: true`, `[ALLOWED BY POLICY]` for `approve(dead, 1000)`, and `[POLICY DENIED]` for 1001, `transfer` and the other address. Without the signer it exits with code 3.
7. **Remove the signer** on the page, rerun with `--expect-removed`, and expect HTTP 401.
8. **Loan protection end to end.** Set `ENABLE_PROTECTION=true` and `NEXT_PUBLIC_ENABLE_PROTECTION=true`, run one backend with workers, borrow against a position, and open **Protect this loan** with a trigger *above* the current health so it acts at once. Then run `scripts/privy/demo-rejection.ts --full` against the live repay policy.

9. Status and limits

Item	Status	Evidence
Login: email, Google, wallet	Working	providers.tsx:18; app settings from `apps.get`
Embedded wallet created at login	Working	Two embedded-wallet users in the database
Embedded wallet signs the user's transactions	Proven for a paymen t	example tx
Access-token verification, 401 on failure	Working	auth/privy.ts:49-73

Item	Status	Evidence
Server acts on a user's wallet through a signer, within the policy	Proven	`03-user-signer.ts` run 2; `0x370e8a29…`; three `policy_violation` refusals
Removing the signer stops the server	Proven	HTTP 401 after `removeSigners`
Repay policy shape	Proven by test	`policies.test.ts` (5 tests); created in Privy by `qa-protection.ts`
Activation checks (signer and allowance)	Proven	Refusal path: `qa-protection.ts`, `SIGNER_MISSING`, rule stays disabled. Success path: the rule went to Protected (screenshot 09) and repaid (section 6)
Full trigger-and-repay cycle on testnet	Proven	Repay `0x882f5ef4…` from the embedded wallet, 2026-10-13 (section 6)
`demo-rejection.ts --full` against a live repay policy	Not exercised	Needs an enabled rule
Embedded wallets only	By design	Signers attach to Privy wallets. External-wallet users see "Loan protection needs a Laxu wallet created with email sign-in." The backend answer `NOT_EMBEDDED_WALLET` is unexercised, because no external-wallet user exists
One active rule per wallet	By design	A signer carries at most one policy, so the backend refuses a second active rule (`ANOTHER_RULE_ACTIVE`)
Policy scope	Per pool, per rule	Each rule's policy names one pool and one per-call cap. It could be tightened further (for example a total limit inside Privy) if Privy's cumulative limits come to support `eth_sendTransaction`
Gas	Paid by the user's wallet	The repay is sent from the user's wallet. Monad bills the gas limit. The worker skips with "Add MON for gas" when the wallet is short
Market speed	Limit	A fast move between 5 s checks can still liquidate. It is a safety net, not a guarantee
Hosted origin in Privy	Set	`allowed_domains`: `https://laxu-monad.vercel.app` and `http://localhost:3000` (testing)
Screenshots	Published	9 in section 7